Assignment 5 — Meal Recipe Search Tool

This notebook adds one real, read-only recipe-search tool to my persona-based meal recommendation project. The public API discovers recipe candidates; the project's deterministic recommendation rules remain responsible for dietary and nutrition checks.

Part A — Tool inventory for the capstone

1. `get_selected_persona_profile`

Purpose, input, output. Reads the profile selected by the application and returns its stated dietary preferences, ingredients to avoid, and meal-level nutrition targets. In the current demonstration it has no input because the application has already selected the profile; it returns a small Python dictionary.

Service and access. This is currently local Python code in `02_Persona_Meal_Recommendation.ipynb`, not a public web service. A production version must be built behind the meal application's private backend/database API. I expose it as code or an MCP tool because the application, rather than the model, should control which profile is selected. It is a read-only operation. The demonstration uses fictional data and no authentication; a production version would use the signed-in application user's authorization. The capstone application owns this data and returns a dictionary/JSON object.

2. `search_recipes_by_name`

Purpose, input, output. Finds real recipe candidates by a recipe-name query. Its input is a non-empty string such as `Arrabiata`; its output is a reduced JSON-like dictionary containing at most five recipe IDs, names, categories, cuisines, and ingredient lists.

Service and access. It calls TheMealDB's public REST endpoint `GET https://www.themealdb.com/api/json/v1/1/search.php?s={recipe_name}`. I use an HTTP API because it is the most direct way to retrieve recipe candidates. It is read-only: it does not modify TheMealDB or my application. The public test endpoint uses the documented development key `1`, so no personal account or API key is involved. TheMealDB owns the service and returns JSON. This tool only discovers candidates; it does not determine nutrition, allergens, or dietary safety.

3. `list_meals`

Purpose, input, output. Returns the meal records available to the prototype, including meal name, dietary tags, ingredients, and configured nutrition fields. It takes no input in the current small example and returns a list of meal dictionaries.

Service and access. This is local Python code in `02_Persona_Meal_Recommendation.ipynb`, not a public web service. A production version must be built on top of a validated food/recipe catalog. Code or MCP is appropriate because an agent needs structured records rather than a webpage. It is read-only. The fictional prototype requires no authentication; a production catalog may require a service API key. The capstone application owns the current demonstration catalog and returns a list/dictionary that can be represented as JSON.

4. `recommend_selected_persona_meals`

Purpose, input, output. Runs deterministic checks over the application's meal candidates using the selected profile. It returns matching candidates plus rejected candidates and explicit reasons.

Service and access. This is local deterministic Python code already implemented in `02_Persona_Meal_Recommendation.ipynb`; a production version must be built as part of the capstone backend. It can be called directly as code or through MCP so an agent can request a result without inventing the decision. It is a read-only computation with no side effect and no authentication in the fictional demonstration. The capstone application owns the logic and returns a dictionary/JSON object.

Data boundary. The public recipe-search tool receives only a recipe-name query. It never receives a persona profile or other user data.

1. Setup

Groq's API is OpenAI-compatible, so we use the `openai` library and point it at Groq. The key is read with `getpass`, so it never appears in the notebook.

In [1]:
```
%pip install -q openai requests

import os, json, getpass
from openai import OpenAI

# Hidden prompt: paste your key when asked. Nothing is stored in the noteboo
GROQ_API_KEY = os.environ.get("GROQ_API_KEY") or getpass.getpass("Groq API k

client = OpenAI(api_key=GROQ_API_KEY, base_url="https://api.groq.com/openai/
MODEL = "openai/gpt-oss-20b"   # supports tool calling (alt: openai/gpt-oss-
```

```
Groq API key: ·········
```

In [2]:
```
# Connection test — you should get a normal reply.
r = client.chat.completions.create(model=MODEL,
        messages=[{"role": "user", "content": "Say hello in one short senter
print(r.choices[0].message.content)
```

```
Hello!
```

If your API needs a key too, read it the same way, never as a literal string:

```
API_KEY = os.environ.get("MY_API_KEY") or getpass.getpass("service
API key: ")
```

2. The loop

`run_agent` from HW3, plus the two guardrails from the graduate extension: tool errors are handed back to the model instead of crashing, and any tool listed in `WRITE_TOOLS` asks you before it runs. Read it; you do not need to change it.

In [3]:
```
TOOLS = {}            # name -> {"schema": ..., "fn": ...}
WRITE_TOOLS = set()   # names of tools that change something -> confirm firs

def run_agent(user_msg, max_steps=6, verbose=True):
    messages = [{"role": "user", "content": user_msg}]
    tool_specs = [t["schema"] for t in TOOLS.values()]
    for _ in range(max_steps):
        r = client.chat.completions.create(model=MODEL, messages=messages, t
        choice = r.choices[0]
        # Append only the fields the API accepts. (Newer openai SDKs add ext
        # `annotations` to model_dump(); Groq rejects those with a 400.)
        entry = {"role": "assistant", "content": choice.message.content}
        if choice.message.tool_calls:
            entry["tool_calls"] = [{"id": c.id, "type": "function",
                                    "function": {"name": c.function.name, "a
                                   for c in choice.message.tool_calls]
        messages.append(entry)

        if choice.finish_reason != "tool_calls":          # exit: it has an
            if verbose: print("FINAL:", choice.message.content)
            return choice.message.content, messages

        for call in choice.message.tool_calls:
            name = call.function.name
            args = json.loads(call.function.arguments)
            if verbose: print(f"CALL   {name}({args})")

            if name in WRITE_TOOLS:                       # checkpoint befo
                ok = input(f"Allow {name}({args})? [y/n] ").strip().lower()
                if ok != "y":
                    result = {"error": "user declined this action"}
                    messages.append({"role": "tool", "tool_call_id": call.id
                    continue

            try:                                          # never let a too
```

```
                result = TOOLS[name]["fn"](**args)
            except Exception as e:
                result = {"error": str(e)}
            if verbose: print(f"RESULT {result}")
            messages.append({"role": "tool", "tool_call_id": call.id, "conte
    return "(max steps reached)", messages
```

Part B — Implement one real tool

The implemented tool is `search_recipes_by_name`. It contacts TheMealDB, a public recipe service, but returns only the small set of fields an agent needs. It is a recipe-discovery tool, not a nutrition or dietary-validation tool.

In [4]:
```
import requests

MEALDB_SEARCH_URL = "https://www.themealdb.com/api/json/v1/1/search.php"
REQUEST_TIMEOUT_SECONDS = 10
```

Step 1 — try the endpoint by hand

In [5]:
```
# Step 1 — hand-test the real endpoint before writing the wrapper.
hand_test = requests.get(
    MEALDB_SEARCH_URL,
    params={"s": "Arrabiata"},
    timeout=REQUEST_TIMEOUT_SECONDS,
)
raw_response = hand_test.json()
print("HTTP status:", hand_test.status_code)
print(json.dumps({
    "idMeal": raw_response["meals"][0]["idMeal"],
    "strMeal": raw_response["meals"][0]["strMeal"],
    "strCategory": raw_response["meals"][0]["strCategory"],
    "strArea": raw_response["meals"][0]["strArea"],
}, indent=2))
# The raw record has many more fields. The wrapper below deliberately returr
```

```
HTTP status: 200
{
  "idMeal": "52771",
  "strMeal": "Spicy Arrabiata Penne",
  "strCategory": "Vegetarian",
  "strArea": "Italian"
}
```

Steps 2, 3, 4 — wrap it, shape the response, handle errors

Checklist: a clear **name** · a **docstring** that says what it does and when to use it · **typed parameters** · a **timeout** · every failure returns `{"error": ...}` · the return value has only the fields the model needs.

In [6]:

```python
# Steps 2–4 — clear name, typed input, timeout, reduced response, and safe e
def search_recipes_by_name(recipe_name: str) -> dict:
    """Find real recipe candidates by name. Use for recipe discovery, not nu
    if not isinstance(recipe_name, str) or not recipe_name.strip():
        return {"error": "recipe_name must be a non-empty string"}
    query = recipe_name.strip()

    try:
        response = requests.get(
            MEALDB_SEARCH_URL,
            params={"s": query},
            timeout=REQUEST_TIMEOUT_SECONDS,
        )
        response.raise_for_status()
        data = response.json()
    except requests.Timeout:
        return {"error": "recipe service timed out; try again"}
    except requests.HTTPError:
        return {"error": f"recipe service returned HTTP {response.status_coc
    except requests.RequestException as error:
        return {"error": f"could not reach recipe service: {error}"}
    except ValueError:
        return {"error": "recipe service returned invalid JSON"}

    if not isinstance(data, dict):
        return {"error": "recipe service returned unexpected data"}

    meals = data.get("meals")
    if not isinstance(meals, list) or not meals:
        return {"error": f"no recipes found for: {query}"}

    recipes = []
    for meal in meals[:5]:
        if not isinstance(meal, dict):
            continue
        ingredients = [
            meal.get(f"strIngredient{i}").strip()
            for i in range(1, 21)
            if isinstance(meal.get(f"strIngredient{i}"), str)
            and meal.get(f"strIngredient{i}").strip()
        ]
        recipes.append({
            "recipe_id": meal.get("idMeal"),
            "name": meal.get("strMeal"),
            "category": meal.get("strCategory") or "not listed",
            "cuisine": meal.get("strArea") or "not listed",
            "ingredients": ingredients,
        })

    if not recipes:
        return {"error": "recipe service returned no usable recipe records"}
    return {"query": query, "recipes": recipes}
```

Step 5 — read or write?

This tool only reads public recipe information. It does not create, update, or delete anything, so no confirmation checkpoint is needed.

In [7]:
```
# Read-only tool: it has no side effects, so this set intentionally stays em
WRITE_TOOLS = set()
```

Step 6 — register it

In [8]:
```
# Step 6 — register the exact wrapper function so the model can see it.
RECIPE_SEARCH_SCHEMA = {
    "type": "function",
    "function": {
        "name": "search_recipes_by_name",
        "description": (
            "Search the public recipe catalog by recipe name. Use only when
            "to find recipe candidates. This does not validate nutrition, al
        ),
        "parameters": {
            "type": "object",
            "properties": {
                "recipe_name": {
                    "type": "string",
                    "description": "The recipe name or recipe-name phrase to
                },
            },
            "required": ["recipe_name"],
        },
    },
}

TOOLS["search_recipes_by_name"] = {
    "schema": RECIPE_SEARCH_SCHEMA,
    "fn": search_recipes_by_name,
}
print("Registered tools:", list(TOOLS))
```

```
Registered tools: ['search_recipes_by_name']
```

4. Tests

Two tests call the wrapper directly; two test the agent loop. Keep every output visible for submission. The agent tests require your own Groq key through the hidden `getpass` prompt above; no key is stored in this notebook.

In [9]:
```
# Unit test 1 — normal input, no model involved.
normal_result = search_recipes_by_name("Arrabiata")
print(json.dumps(normal_result, indent=2))
```

```
{
  "query": "Arrabiata",
  "recipes": [
    {
      "recipe_id": "52771",
      "name": "Spicy Arrabiata Penne",
      "category": "Vegetarian",
      "cuisine": "Italian",
      "ingredients": [
        "penne rigate",
        "olive oil",
        "garlic",
        "chopped tomatoes",
        "red chilli flakes",
        "italian seasoning",
        "basil",
        "Parmigiano-Reggiano"
      ]
    }
  ]
}
```

In [10]:
```
# Unit test 2 — a query with no result. It must return an error dictionary,
no_result = search_recipes_by_name("xqznotarealmeal987")
print(json.dumps(no_result, indent=2))
assert "error" in no_result
```

```
{
  "error": "no recipes found for: xqznotarealmeal987"
}
```

In [11]:
```
# LLM test 1 — this SHOULD trigger the recipe-search tool. Watch for CALL /
answer, messages = run_agent("Find recipe candidates named Arrabiata.")
```

```
CALL   search_recipes_by_name({'recipe_name': 'Arrabiata'})
RESULT {'query': 'Arrabiata', 'recipes': [{'recipe_id': '52771', 'name': 'Sp
icy Arrabiata Penne', 'category': 'Vegetarian', 'cuisine': 'Italian', 'ingre
dients': ['penne rigate', 'olive oil', 'garlic', 'chopped tomatoes', 'red ch
illi flakes', 'italian seasoning', 'basil', 'Parmigiano-Reggiano']}]}
FINAL: Here are the recipe candidates that match “Arrabiata” in our catalog:

| Recipe ID | Name | Category | Cuisine |
|-----------|------|----------|---------|
| 52771 | **Spicy Arrabiata Penne** | Vegetarian | Italian |

*This recipe includes penne rigate, olive oil, garlic, chopped tomatoes, red
chili flakes, Italian seasoning, basil, and Parmigiano-Reggiano.*

If you’d like more details (full ingredient list, instructions, nutrition, a
llergens, etc.) or if you’d like to explore similar variations (e.g., Arrabi
ata with shrimp, arrabiata sauce for pasta, etc.), just let me know!
```

In [12]:
```
# LLM test 2 — this should NOT trigger recipe search. There should be a FINA
answer, messages = run_agent("What does JSON stand for?")
```

```
FINAL: JSON stands for **JavaScript Object Notation**.
```

(Optional) print the full message trace of the last run:

In [13]:
```
for m in messages:
    print(m.get("role"), "->", m.get("content") or m.get("tool_calls"))
```

```
user -> What does JSON stand for?
assistant -> JSON stands for **JavaScript Object Notation**.
```

5. Graduate Extension — the same tool as an MCP server

Undergraduates: leave this section empty.

FastMCP turns your function into an MCP tool with one line. The client below runs **in this notebook**, so nothing else has to be installed or connected: it asks the server for its tool list (the server describes itself: name, description, parameters) and then calls the tool through the protocol.

In [14]:
```
%pip install -q fastmcp==4.0.3
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0.0/859.7 kB ? eta -:--:--
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 849.9/859.7 kB 33.5 MB/s eta 0:0
0:01
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 859.7/859.7 kB 18.5 MB/s eta 0:0
0:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 279.4/279.4 kB 16.7 MB/s eta 0:0
0:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 169.4/169.4 kB 11.8 MB/s eta 0:0
0:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 365.7/365.7 kB 23.4 MB/s eta 0:0
0:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 69.1/69.1 kB 4.1 MB/s eta 0:00:0
0
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 96.4/96.4 kB 7.0 MB/s eta 0:00:0
0
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 171.3/171.3 kB 11.9 MB/s eta 0:0
0:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 69.4/69.4 kB 4.9 MB/s eta 0:00:0
0
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 279.3/279.3 kB 16.8 MB/s eta 0:0
0:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 162.6/162.6 kB 11.4 MB/s eta 0:0
0:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 331.1/331.1 kB 22.7 MB/s eta 0:0
0:00
```

In [15]:
```
from fastmcp import FastMCP

mcp = FastMCP("Meal Recipe Search Server")

# Register the exact Part B wrapper. FastMCP derives its name, description,
mcp.tool(search_recipes_by_name)
```

Out[15]:

search_recipes_by_name

def search_recipes_by_name(recipe_name: str) -> dict

Find real recipe candidates by name. Use for recipe discovery, not nutrition, allergen, or dietary validation.

In [17]:

```python
from fastmcp import Client

async with Client(mcp) as c:
    tools = await c.list_tools()
    for t in tools:
        print("name:       ", t.name)
        print("description:", t.description)
        print("parameters: ", json.dumps(getattr(t, "input_schema", None) or
    result = await c.call_tool("search_recipes_by_name", {"recipe_name": "Ar
    print("call result:", result.content[0].text)
```

```
name:        search_recipes_by_name
description: Find real recipe candidates by name. Use for recipe discovery,
not nutrition, allergen, or dietary validation.
parameters:  {"type": "object", "additionalProperties": false, "properties":
{"recipe_name": {"type": "string"}}, "required": ["recipe_name"]}
call result: {"query":"Arrabiata","recipes":[{"recipe_id":"52771","name":"Sp
icy Arrabiata Penne","category":"Vegetarian","cuisine":"Italian","ingredient
s":["penne rigate","olive oil","garlic","chopped tomatoes","red chilli flake
s","italian seasoning","basil","Parmigiano-Reggiano"]}]}
```

Optional, +1: connect it to a real host. Save the server cell as `server.py` with `mcp.run()` at the end, then add it to your host's config (Claude Desktop → *Settings* → *Developer* → *Edit Config*; VS Code / Cursor have an equivalent):

```
{"mcpServers": {"meal-recipe-search": {"command": "python", "args":
["/full/path/to/server.py"]}}}
```

Restart the host and screenshot your tool in its tool list.

AI use & verification note

1. **AI tools I used, and what for:** I used an AI coding assistant to help interpret the assignment, draft the TheMealDB wrapper and function schema, and check the notebook structure.
2. **How I checked the output:** I hand-tested TheMealDB's endpoint with `Arrabiata`, ran the direct-function normal and no-result tests, and inspected the agent trace and MCP `list_tools` / `call_tool` outputs.
3. **Any correction or adjustment I made:** I reduced the raw API response to recipe ID, name, category, cuisine, and ingredients; I added timeout and error handling; and I explicitly limited the tool to candidate discovery rather than nutrition or dietary validation.